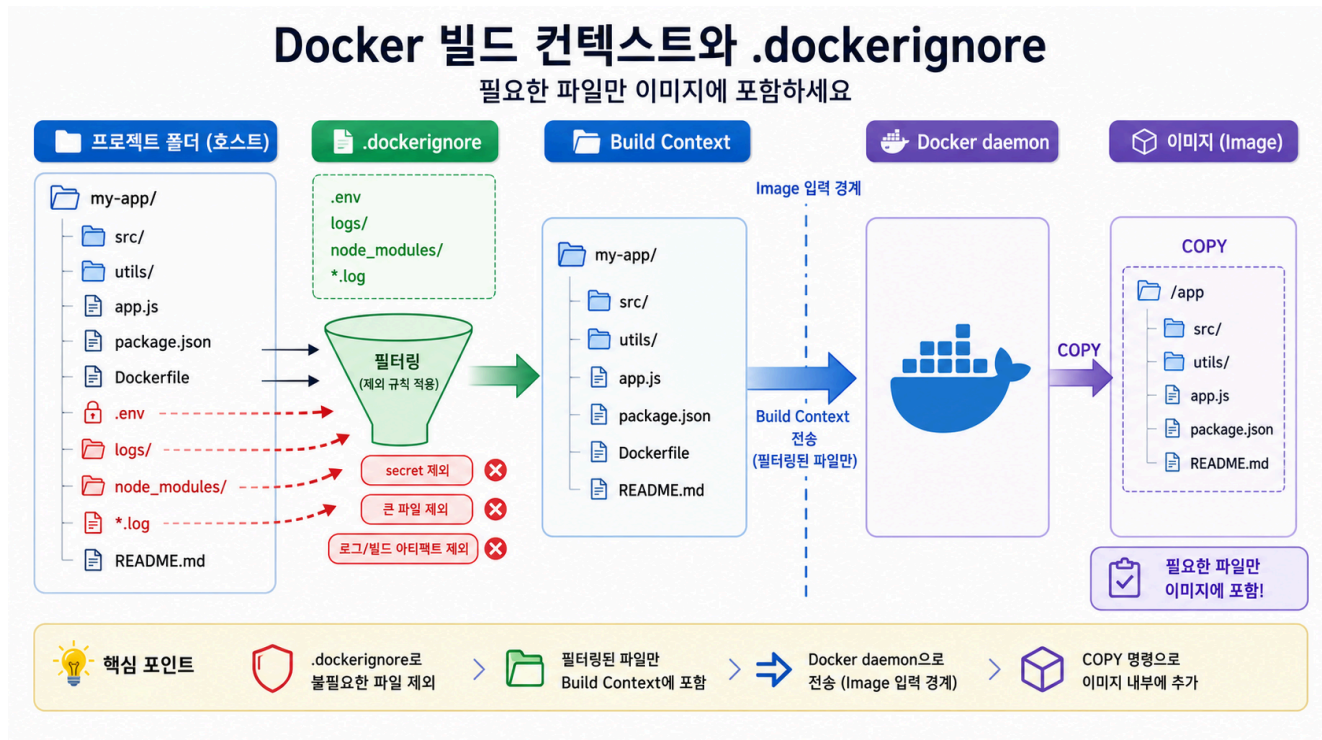


3교시: Build context gate - image에 들어가면 안 되는 것 막기



수업 목표

- build context가 Docker daemon에 전달되는 입력임을 설명한다.
- .dockerignore로 secret, log, dependency/cache directory를 제외한다.
- COPY . .와 필요한 파일만 복사하는 방식의 위험 차이를 구분한다.

개념 설명

docker build -t name .에서 마지막 .은 그냥 장식이 아니다. 이 .이 build context다. Docker는 이 directory 안의 파일을 build 입력으로 보고, Dockerfile의 COPY는 이 context 안에서만 source를 찾는다.

그래서 build context는 두 가지 위험을 만든다. 첫째, 필요한 파일이 context 밖에 있으면 COPY failed가 난다. 둘째, .env, token, log, Python cache, node_modules, dist, build 같은 결과물이 context 안에 있으면 image에 들어가거나 build가 느려질 수 있다. .dockerignore는 이 입력 경계를 정리하는 gate다.

.dockerignore는 .gitignore와 목적이 비슷해 보이지만 대상이 다르다. .gitignore는 Git commit에서 제외할 파일을 정하고, .dockerignore는 Docker build context에서 제외할 파일을 정한다. Git에는 올리지 않아도 되는 파일과 Docker image에 들어가면 안 되는 파일이 겹치는 경우가 많지만, 둘은 별도로 관리해야 한다.

.dockerignore 예시와 해석

현재 lab의 .dockerignore는 다음 종류를 제외한다.

```
.git
*.log
.env
.env.*
__pycache__/
*.pyc
.pytest_cache/
node_modules/
dist/
build/
coverage/
tmp/
```

패턴	제외 이유
.git	Git history와 내부 metadata는 image 실행에 필요 없다. context만 커진다.
*.log	실행 로그는 build 산출물이 아니다. 이전 실행 흔적이 image에 들어가면 안 된다.
.env, .env.*	password, token, API key 같은 secret이 들어갈 수 있다.
__pycache__/, *.pyc, .pytest_cache/	Python 실행/테스트 cache다. source가 아니고 재생성 가능하다.
node_modules/	Node dependency directory는 매우 크고 host OS에 묶일 수 있다. 보통 image 안에서 install하거나 별도 전략을 쓴다.
dist/, build/	frontend/backend build 결과물이다. 의도적으로 복사할 때만 포함한다.
coverage/	test coverage 결과물이다. image 실행에는 필요 없다.
tmp/	임시 파일이다. 재현성과 보안에 방해가 된다.

실습 명령

```
cd week2/day3/labs/static-site
find . -maxdepth 2 -type f | sort
sed -n '1,160p' .dockerignore
du -sh .
```

위험 재현

```
mkdir -p __pycache__ node_modules dist build coverage tmp
printf "DO_NOT_COMMIT_TOKEN=example" > .env
printf "debug log" > app.log
printf "cache" > __pycache__/app.cpython-312.pyc
printf "large dependency placeholder" > node_modules/example.txt
printf "compiled output" > dist/app.bundle.js
find . -maxdepth 2 -type f | sort
sed -n '1,160p' .dockerignore
```

Expected interpretation:

위 파일들은 lab 실행에 필요하지 않다.
 .dockerignore가 없거나 약하면 build context에 포함될 수 있다.
 .env는 보안 위험, node_modules/dist/build/cache는 크기와 재현성 위험이다.

정리:

```
rm -rf .env app.log __pycache__ node_modules dist build coverage tmp
```

판단 기준

증상	첫 확인 위치
COPY failed	Dockerfile source path와 build context 위치
image/context가 과하게 큼	du -sh ., .dockerignore
secret 포함 위험	.env, .env.*, token, credential 파일 존재 여부
Python cache 포함	__pycache__, *.pyc, .pytest_cache
Node dependency 포함	node_modules
build output 포함	dist, build, coverage
build가 느림	context 크기와 dependency/cache 포함 여부

핵심 포인트

build context는 image로 보낼 수 있는 입력 경계다. build가 성공해도 context gate가 없으면 보안과 재현성에서 실패할 수 있다.

다음 연결

다음 교시는 context를 통과한 앱을 실제 image로 만들고 HTTP acceptance check를 남긴다.